LAB MANUAL

Charotar University of Science and Technology (CHARUSAT)

Chandubhai S Patel Institute of Technology (CSPIT)

Smt. Kundanben Dinsha Patel Department of Information Technology

IT443 Language Processor

A.Y. 2019-20

Practical List

Sr. No.	Aim of the Practical	Hours	COs	POs	PEOs
1.	Write a 'C' program and generate the following codes for the program. a. Preprocessed code b. Assembly Code c. Object Code d. Executable Code	02	1	1,2	1
2.	Use Macro features of C language and demonstrate the following types of macro with example. 1) Simple Macro 2) Macro with Argument 3) Nested Macro	02	1	1,2	2,3,4
3.	Write a Lexical Analyzer using Lex or Flex utility of UNIX for following: 1. A lexer to print out all numbers from a given file. 2. A lexer which adds line numbers to the given file and display the same onto the standard output. 3. A lexer which attempt to extract only comments from a C program and display the same on standard output. 4. A lexer to do the word count functions of the wc command in UNIX. It prints the number of lines, words and characters in a file. 5. A lexer which classifies tokens as words, numbers or "other". 6. Write a Lex Program to count number of vowels and consonants.	06	1,2	1,2	2,3
4.	Implement Lexical analyzer for C language using lex.	04	1,2	1,2,3	3,4
5.	Explore the JFLAP Tool. Demonstrate the finite automata in the Tool.	04	1,2	1,2	2,3
6.	Write a program which enters Transition Table, accepting state and input string as input and checks whether the given string is accepted or not.	02	3	1,2,3	3,4
7.	Write Generation Of Three Address Code in C Programming.	02	3	1,2,3	2,3,4
8.	Write a Parser using yacc or utility of UNIX for following: Write a Program for a simple desk calculator using YACC Specification.	04	3	3	2,3,4
9.	Demonstrate the parsing technique using ANTLR Tool.	04	3	3	2,3,4

Practical 1

Aim: Write a 'C' program and generate the following codes for the program.

a. Preprocessed code
b. Assembly Code
c. Object Code
d. Executable Code

Software Required:

- Turbo C Compiler

Knowledge Required:

- Concepts of C Programming

Theory/Logic:

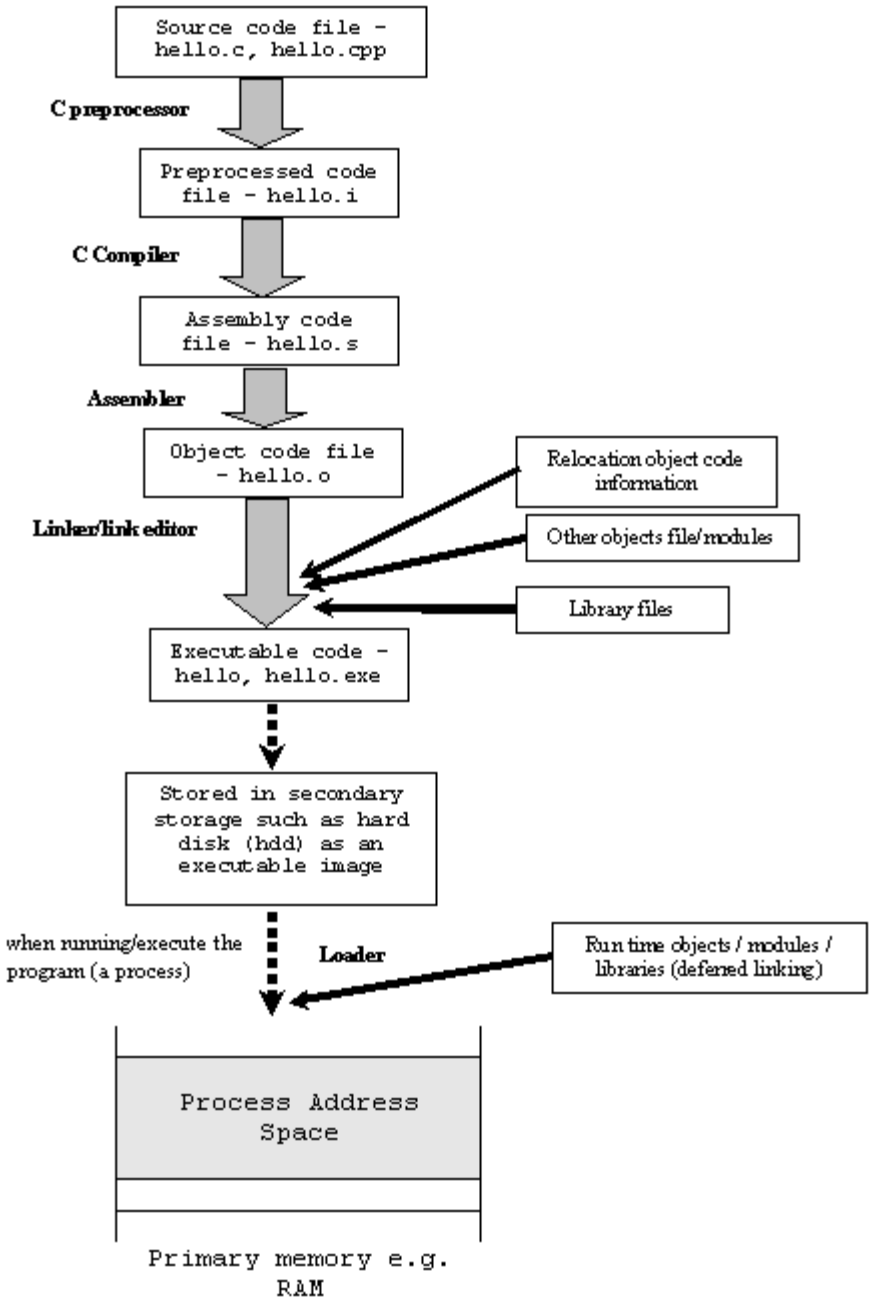
The compilation and execution process of C can be divided in to multiple steps:

- Preprocessing - Using a Preprocessor program to convert C source code in expanded source code. "#includes" and "#defines" statements will be processed and replaced actually source codes in this step.
- Compilation - Using a Compiler program to convert C expanded source to assembly source code.
- Assembly - Using a Assembler program to convert assembly source code to object code.
- Linking - Using a Linker program to convert object code to executable code. Multiple units of object codes are linked to together in this step.
- Loading - Using a Loader program to load the executable code into CPU for execution.
-

Here is a simple table showing input and output of each step in the compilation and execution process:

```
Input                  Program        Output

source code          > Preprocessor > expanded source code
expanded source code > Compiler     > assembly source code
assembly code        > Assembler    > object code
object code          > Linker       > executable code
executable code      > Loader       > execution
```



Source code :

Output:

Different compilation and execution steps on a Linux system:

- "cpp hello.c -o hello.i" - Preprocessor preprocessing hello.c and saving output to hello.i.
- "cc1 hello.i -o hello.s" - Compiler compiling hello.i and saving output to hello.s.
- "as hello.s -o hello.o" - Assembler assembling hello.s and saving output to hello.o.
- "ld hello.o -o hello" - Linker linking hello.o and saving output to hello.
- "load hello" - Loader loading hello and running hello.

Advantages:

- Using the step by step process shown above, we can view all the intermediate form which are generated when we convert source program to target program

Application:

- Preprocessed or Assembly code which is used by assembler or other application

Conclusion:

- This experiment is useful to get all the output from all language processor (preprocessor, assembler, compiler, linker/loader) for a given source program.

Question:

1. What is the output of preprocessor?
2. What is the use of linker/loader in language processing system?

Practical 2

Aim: Use Macro features of C language and demonstrate the following types of macro with example.

1) Simple Macro 2) Macro with Argument 3) Nested Macro

Software Required:

- Turbo C Compiler

Knowledge Required:

- Concepts of Prprocessor

Theory:

The C preprocessor is a macro preprocessor (allows you to define macros) that transforms your program before it is compiled. These transformations can be inclusion of header file, macro expansions etc.

All preprocessing directives begins with a # symbol. For example,

#define PI 3.14

The #include preprocessor is used to include header files to a C program. For example,

#include <stdio.h>

Here, "stdio.h" is a header file. The #include preprocessor directive replaces the above line with the contents of stdio.h header file which contains function and macro definitions.

Program:

Simple Macro:

```
#include<stdio.h>
#include<math.h>
#define pi 3.14
#define mul(a) ((a)*(a))
void main()
{
float b;
b=pi +mul(5);
printf("The area of circle:: %f",b);
```

```
}
```

Nested macro:

```
#include<stdio.h>
#define sum(x,y,z) ((x)+(y)+(z))
#define avg(x,y,z) ((sum(x,y,z))/(2))
void main()
{
int c;
c=avg(4,6,8);
printf("avg== %d",c);
}
```

Argumented macro:

```
#include<stdio.h>
#define max(x,y) (((x)>(y)) ? ((x)=(y)))
void main()
{
int a;
a=max(78,90);
printf("Maximum::%d",a);
```

Inline Macro:

```
#include<stdio.h>
#define sumarray(a) for(i=0;i<10;i++)
{ sum=sum+a[i]; printf("Sum is::%d",sum);
}
void main()
{
int a[10]={1,2,3,4,5,6,7,8,9,10},;
sum=sumarray(a);
}
```

Question:

1. **What is the role of Preprocessor?**

.

Practical 3

Aim: Write a Lexical Analyzer using Lex or Flex utility of UNIX for following:

1. A lexer to print out all numbers from a given file.
2. A lexer which adds line numbers to the given file and display the same onto the standard output.
3. A lexer which attempt to extract only comments from a C program and display the same on standard output.
4. A lexer to do the word count functions of the wc command in UNIX. It prints the number of lines, words and characters in a file.
5. A lexer which classifies tokens as words, numbers or "other".
6. Write a Lex Program to count number of vowels and consonants.

Software Required:

- Lex or Flex

Knowledge Required:

- knowledge of writing regular expressions

Theory/Logic:

- Flex is a tool for generating scanners: programs which recognized lexical patterns in text.
- Flex reads the given input files, or its standard input if no file names are given, for a description of a scanner to generate.
- The description is in the form of pairs of regular expressions and C code, called rules.
- Flex generates as output a C source file, lex.yy.c, which defines a routine yylex(). This file is compiled and linked with the -lfl library to produce an executable.
- When the executable is run, it analyzes its input for occurrences of the regular expressions. Whenever it finds one, it executes the corresponding C code.

Format of the Input File:

- The Flex input file consists of three sections, separated by a line with just %% in it:

```
definitions
%%
rules
%%
```

user code

- The definitions section contains declarations of simple name definitions to simplify the scanner specification, and declarations of start conditions, which are explained in a later section.

- The rules section of the flex input contains a series of rules of the form:

  pattern action

  where the pattern must be unintended and the action must begin on the same line.

Source Code:

1. **A lexer to print out all numbers in a file**

```
%{
}%
m   [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit}|_)*
number {digit}+(\.{digit}+)?(E[+\-]?{digit}+)?

%%

{id}     ;
{number}    {printf("%s is NUMBER",yytext);}

 %%
```

2. **Lexer to print out all HTML tags in a file.**

```
%{
}%
letter      [a-z A-Z]
word        {letter}*
start tag   [<]({letter})+[>]
end tag     [<][|]({letter}) +[>]
%%
{start tag} {printf("%s is start tag",yytext);}
{end tag}   {printf("%s is end tag",yytext);}
```

3. A lexer which adds line numbers to the given file

```
%{
#include<stdio.h>
int yylineno;
%}
%%
^(.*)\n {printf("%d\t%s",++yylineno,yytext);}
. ;
\n ;
%%
int main(void) {
        yylex();
        printf("\n");
        return 0; }
```

4. A lexer which attempt to extract only comments...

```
%{
#include<stdio.h>
%}
%%
"//".*\n {printf("%s ",yytext);}
"/*".*\n*.*"*/" {printf("%s ",yytext);}
. ;
\n ;
%%
int main(void)
{
        yylex();
        printf("\n");
        return 0;
}
```

5. A lexer which replaces all the occurrence of "hi" with "HI" and "hello" with "HELLO".

```
%{
#include<stdio.h>
%}
%%
"hi" {printf("HI");}
"hello" {printf("HELLO");}
%%
int main(void){
        yylex();
        printf("\n");
        return 0;
```

```
}
```

6. A lexer to do word count function of wc command in UNIX.It prints the number of lines,words and characters in a file.

```
%{
int c=0;
int a=0;
int b=0;
%}
delim  [ \t]
d    [\n]
letter [a-zA-Z]
digit  [0-9]
ws   {delim}+

%%

{digit}       {b++;printf("char=%d\n",b);}
{letter}      {b++;printf("char=%d\n",b);}
{ws}          {c++;printf("word=%d\n",c);}
{d}           {a++;printf("no of lines=%d\n",a);}
     .        {printf("in valid");}
%%
```

7. Classifying tokens as words, numbers or others.

```
%{
delim  [ \t\n]
ws   {delim}+
letter [A-Za-z]
digit  [0-9]
id   {letter}({letter}|{digit}|_)*
literal ["\']({digit}|{letter}|_)*["\']
number {digit}+(\.{digit}+)?(E[+\-]?{digit}+)?
%}
%%
{ws}      ;
"int"|"char"|"float"|"void"    {printf("%s is KEYWORD\n",yytext);}
{literal}     {printf("%s is LITERAL \n",yytext);}
{id}       {printf("%s is IDENTIFIER \n",yytext);}
{number}     {printf("%s is NUMBER \n",yytext);}
"%"|"+"|"-"|"*"|"="      {printf("%s is OPERATOR \n",yytext);}
";"|"#"|"$"|"%"      {printf("%s is SPECIAL SYMBOL\n",yytext);}
.        {printf("%s is INVALID SYMBOL\n",yytext);}
%%
```

8. A lexer that changes all numbers to hexadecimal in input file while ignoring all others.

```
%{
#include<stdio.h>
#include<math.h>
int i=0,j;
int n;
int r[20]={0};
char hex[6]={'A','B','C','D','E','F'};
%}
%%
[0-9]*  {
        n=atoi(yytext);
        while(n>0)
        {
                r[i]=n%16;
                n=n/16;
                i++;
        }
        for(j=i-1;j>=0;j--)
        {
                if( r[j] > 9 && r[j] < 16)
                {
                        n= r[j] % 10;
                        printf("%c",hex[n]);
                }
                else
                        printf("%d",r[j]);
        }
        printf("\n");
        for(i=0;i<20;i++)
                r[i]=0;
        n=0;
        i=0;
        j=0;
        }

.       {printf("%s",yytext);}
%%
int main(void)
{
        yylex();
        printf("\n");
        return 0;
}
```

9. This lexer prints only words followed by punctuation. If the following sentence was the input from standard input:
"hello", they said.

how are you? I cannot tell.
It will print the words hello, said, you, and tell. It will not print the punctuation; only the words.

```
%{
#include<stdio.h>
#include<string.h>
%}
%%
[a-zA-Z0-9]+/[,?!;.'"] {
        printf("%s ",yytext);
        }
. ;
%%
int main(void)
{
        yylex();
        printf("\n");
        return 0;
}
```

Advantages:

- It makes the lexical analysis very simple and easy.
- It can identifies HTML tag and display it.
- It can also identifies special symbol.
- It can classifies Word ,Numbers ..

Application:

- The application is used to carry out the lexical analysis.

Conclusion:

- The application successfully carries the lexical analysis.

Question:

1. **What is the syntax of the input file in lex?**

   → definitions
   %%
   rules
   %%
   user code

Practical 4

Aim: Implement Lexical analyzer for C language using lex.

Software Required:

- Lex or Flex

Knowledge Required:

- knowledge of writing regular expressions

Source code:

```
%{
#include<stdio.h>
%}
digit [0-9]
%option yylineno
%%

"include"|"int"|"char"|"float"|"void"|"main"|"for"|"while"|"if"|"else"|"stdio.h"
|"%d"|"%s"|"%c"|"printf"|"scanf"        {printf("<Keyword>:%s\n",yytext);}

[a-zA-Z][a-zA-Z0-9]*          {printf("<Identifier>:%s\n",yytext);}
[;{}().?:!,]                  {printf("<Punctuation>:%s\n",yytext);}
{digit}+                      {printf("<Constant>:%s\n",yytext);}
{digit}+(.{digit}+)           {printf("<Floating point digit>:%s\n",yytext);}
[-%+*=<>/]                    {printf("<Operator>:%s\n",yytext);}
"//".*                        {printf("<Single-line comment>:%s\n",yytext);}
"/*"(.|\n)+"*/"               {printf("<Multi-line comment>:%s\n",yytext);}
[#&"]                         {printf("<Special Symbol>:%s\n",yytext);}
[ \t]                         ;
.       {printf("<ERROR>:%s is not valid Token at line:%d\n",yytext,yylineno);}

%%
main()
{
yylex();
}
```

Conclusion:

- This experiment generates the output of lexical analyzer phase for a given ‘C’ program in Lex/Flex utility

Question:

1. What is the use of “.” in rule section of LEX file?

Practical 5

Aim: Explore the JFLAP Tool. Demonstrate the finite automata in the Tool.

Software Required:

- JFLAP Tool

Knowledge Required:

- Concepts of Finite Automata

Theory/Logic:

JFLAP is software for experimenting with formal languages topics including nondeterministic finite automata, nondeterministic pushdown automata, multi-tape Turing machines, several types of grammars, parsing, and L-systems. In addition to constructing and testing examples for these, JFLAP allows one to experiment with construction proofs from one form to another, such as converting an NFA to a DFA to a minimal state DFA to a regular expression or regular grammar.

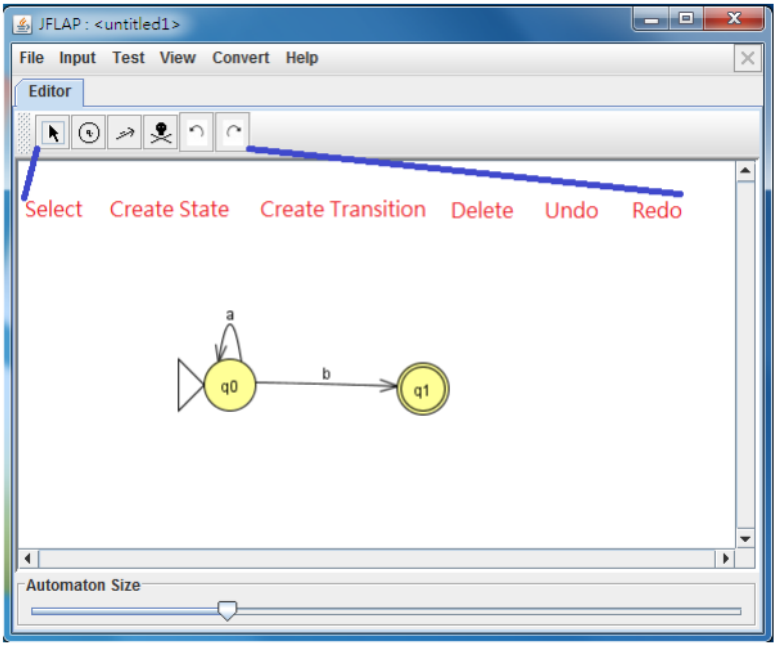
JFLAP is a package of graphical tools which can be used as an aid in learning the basic concepts of Formal Languages and Automata Theory.

http://www.jflap.org/

You need Java runtime environment (JRE) – http://www.java.com • Execute JFLAP – Create finite automaton

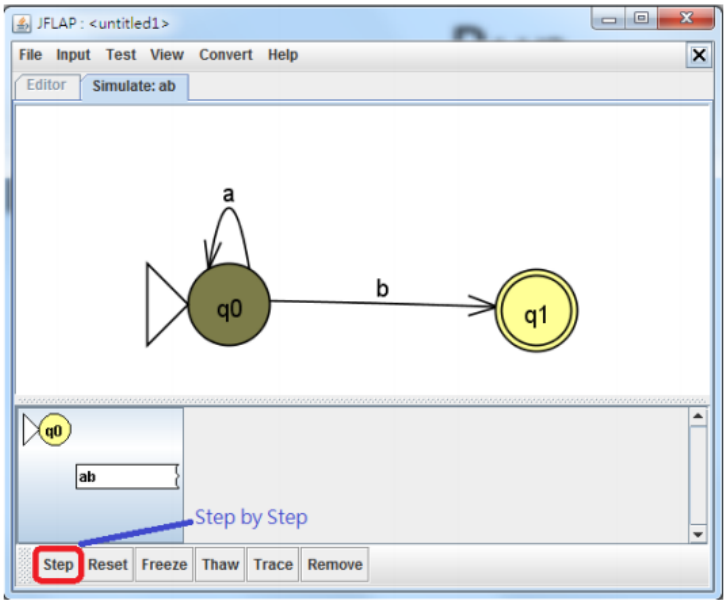
Create Finite Automata:

- Select
  - Right click on states to popup property menu
    - Initial state
    - Final state
- Create state & Delete item
  - Single click
- Create Transition
  - Drag and drop between states
  - Input alphabets

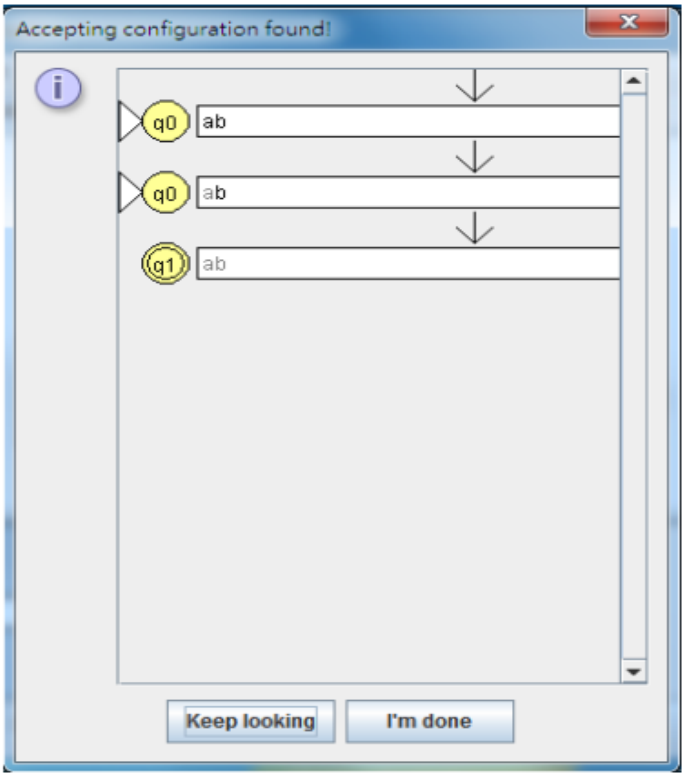


RUN:

- Input Menu
  - Step by State
  - Input “ab”

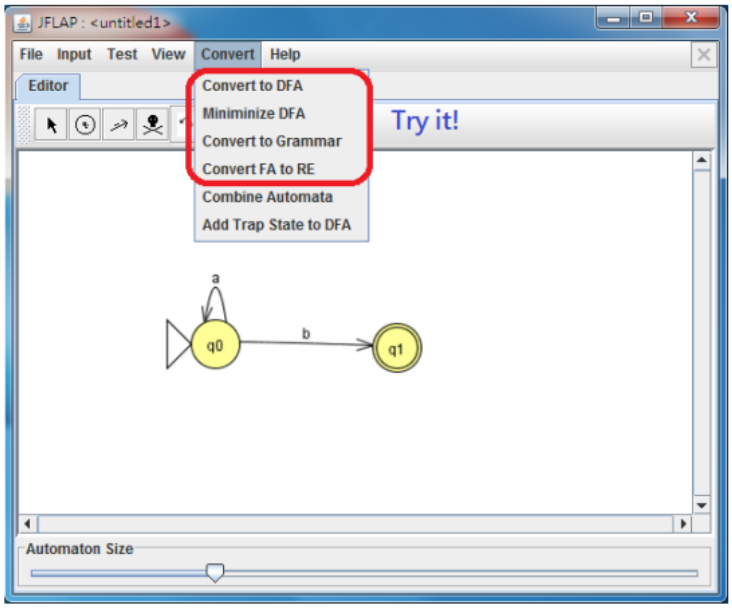


- Input Menu
  - Fast Run
  - Input “ab”



Automata Conversation:

- Try it
  - To DFA
  - Minimize
  - To grammar
  - To RE
- Verify your answer



References:

- JFLAP Tutorial – http://www.jflap.org/tutorial/
- JFLAP sample file – http://www.jflap.org/samplefiles.html

Question:

1. **Design RE for the valid CHARUSAT Student-ID and prepare the minimized DFA in Jflap Tool.**

Practical 6

Aim: Write a program which enters Transition Table, accepting state and input string as input and checks whether the given string is accepted or not.

Software Required:

- Turbo C Compiler

Knowledge Required:

- Finite Automata concepts

Theory/Logic:
Finite Automata
A recognizer for a language is a program that takes a string x as an input and answers "yes" if x is a sentence of the language and "no" otherwise.

One can compile any regular expression into a recognizer by constructing a generalized transition diagram called a finite automation.

A finite automation can be deterministic means that more than one transition out of a state may be possible on a same input symbol.
Both automata are capable of recognizing what regular expression can denote.

Nondeterministic Finite Automata (NFA)
A nondeterministic finite automation is a mathematical model consists of

1. a set of states S;
2. a set of input symbol, $\sum$, called the input symbols alphabet.
3. a transition function move that maps state-symbol pairs to sets of states.
4. a state so called the initial or the start state.
5. a set of states F called the accepting or final state.

Deterministic Finite Automata (DFA)
A deterministic finite automation is a special case of a non-deterministic finite automation (NFA) in which

1. no state has an $\in$-transition
2. for each state s and input symbol a, there is at most one edge labeled a leaving s.

Algorithm for Simulating a DFA

INPUT:

- string x
- a DFA with start state, so . . .
- a set of accepting state's F.

OUTPUT:

- The answer 'yes' if D accepts x; 'no' otherwise.

The function move (S, C) gives a new state from state s on input character C.
The function 'nextchar' returns the next character in the string.
Initialization:
S := S_0
C := nextchar;
while not end-of-file do
S := move (S, C)
C := nextchar;
If S is in F then
return "yes"
else
return "No".

Source Code:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

struct trans_table
{
      char prev_state;
      char symbol;
      char next_state;
}t[50];


void main()
{
      char curr_state,acc_state;
      int i,j,no_rows,no_acc;
      char str[50];

      clrscr();
      printf("Enter no. of rows in transition table: ");
      scanf("%d",&no_rows);
      for(i=0;i<no_rows;i++)
      {
            flushall();
            scanf("%c",&t[i].prev_state);
            flushall();
            scanf("%c",&t[i].symbol);
            flushall();
            scanf("%c",&t[i].next_state);
      }

      curr_state=t[0].prev_state;
      printf("\nEnter the string u want to check: ");
      scanf("%s",str);
       printf("Enter accepting state");
      flushall();
      scanf("%c",&acc_state);
```

```
	i=0;
	printf("Accpt state is %c",acc_state);
	while(str[i]!='\0')
	{
		for(j=0;j<no_rows;j++)
		{
			if(t[j].prev_state==curr_state)
			{
				if(t[j].symbol==str[i])
				{
					curr_state=t[j].next_state;
					i++;
					break;
				}
			}
		}

		if(j==no_rows)
		{
			printf("Invalid character in input");
			exit(0);
		}
	}
	if(curr_state==acc_state)
		printf("\n string is valid");
	else
		printf("\n Invalid string");
	getch();
}
```

Advantages:

- 'Deterministic' refers to the uniqueness of the computation. so DFAs "arrive at the answer" faster than equivalent NFAs.
- provides fast access to the transitions of given state on given input character.

Disadvantages:

- Take more lot of space when input is large most transitions are to the empty set.

Application:

Conclusion:

Question:

1. Differentiate DFA and NFA.

Advantages:

- It makes the lexical analysis very simple and easy.
- It can identifies HTML tag and display it.
- It can also identifies special symbol.
- It can classifies Word ,Numbers ..

Application:

- The application is used to carry out the lexical analysis.

Conclusion:

- The application successfully carries the lexical analysis.

Question:

1. Write the regular expression for IDENTIFIER.
→ ID [a-z][a-z0-9]*

2. Write lex program which counts occurrences of vowels in the given file.

Practical 7

Aim: Write Generation Of Three Address Code in C Programming.

Software Required:

- Turbo C

Knowledge Required:

- **Knowledge for Intermediate Code Generation concepts.**

Theory/Logic:

- Three-address code is an intermediate code. It is used by the optimizing compilers.
- In three-address code, the given expression is broken down into several separate instructions. These instructions can easily translate into assembly language.
- Each Three address code instruction has at most three operands.
- The reason for term "Three address code" is that each statement usually contains three addresses, two for operands and one for the result.

Example:
X = a * b + c * d – e *f

t1 = a * b
t2 = c * d
t3 = e * f
t4 = t1 + t2
t5 = t4 – t3
X = t5

Source Code:

```
#include<stdio.h>
#include<string.h>
void pm();
void plus();
void div();
int i,ch,j,l,addr=100;
char ex[10], exp[10] ,exp1[10],exp2[10],id1[5],op[5],id2[5];
void main()
{
clrscr();
while(1)
{
printf("\n1.assignment\n2.arithmetic\n3.relational\n4.Exit\nEnter the choice:");
scanf("%d",&ch);
switch(ch)
```

```
{
case 1:
printf("\nEnter the expression with assignment operator:");
scanf("%s",exp);
l=strlen(exp);
exp2[0]='\0';
i=0;


while(exp[i]!='=')
{
i++;
}
strncat(exp2,exp,i);
strrev(exp);
exp1[0]='\0';
strncat(exp1,exp,l-(i+1));
strrev(exp1);
printf("Three address code:\ntemp=%s\n%s=temp\n",exp1,exp2);
break;

case 2:
printf("\nEnter the expression with arithmetic operator:");
scanf("%s",ex);
strcpy(exp,ex);
l=strlen(exp);
exp1[0]='\0';

for(i=0;i<l;i++)
{
if(exp[i]=='+'||exp[i]=='-')
{
if(exp[i+2]=='/'||exp[i+2]=='*')
{
pm();
break;
}
else
{
plus();
break;
}
}
else if(exp[i]=='/'||exp[i]=='*')
{
div();
```

```
break;
}
}
break;

case 3:
printf("Enter the expression with relational operator");
scanf("%s%s%s",&id1,&op,&id2);
if(((strcmp(op,"<")==0)||(strcmp(op,">")==0)||(strcmp(op,"<=")==0)||(strcmp(op,">=")=
=0)||(strcmp(op,"==")==0)||(strcmp(op,"!=")==0))==0)
printf("Expression is error");
else
{
printf("\n%d\tif %s%s%s goto %d",addr,id1,op,id2,addr+3);
addr++;
printf("\n%d\t T:=0",addr);
addr++;
printf("\n%d\t goto %d",addr,addr+2);
addr++;
printf("\n%d\t T:=1",addr);
}
break;
case 4:
exit(0);
}
}
}
void pm()
{
strrev(exp);
j=l-i-1;
strncat(exp1,exp,j);
strrev(exp1);
printf("Three address code:\ntemp=%s\ntemp1=%c%ctemp\n",exp1,exp[j+1],exp[j]);
}
void div()
{
strncat(exp1,exp,i+2);
printf("Three address code:\ntemp=%s\ntemp1=temp%c%c\n",exp1,exp[i+2],exp[i+3]);
}
void plus()
{
strncat(exp1,exp,i+2);
printf("Three address code:\ntemp=%s\ntemp1=temp%c%c\n",exp1,exp[i+2],exp[i+3]);
}
```

Example Generation of Three Address Project Output Result

1. assignment
2. arithmetic
3. relational
4. Exit
Enter the choice:1
Enter the expression with assignment operator:
a=b
Three address code:
temp=b
a=temp

1.assignment
2.arithmetic
3.relational
4.Exit
Enter the choice:2
Enter the expression with arithmetic operator:
a+b-c
Three address code:
temp=a+b
temp1=temp-c

1.assignment
2.arithmetic
3.relational
4.Exit
Enter the choice:2
Enter the expression with arithmetic operator:
a-b/c
Three address code:
temp=b/c
temp1=a-temp

1.assignment
2.arithmetic
3.relational
4.Exit
Enter the choice:2
Enter the expression with arithmetic operator:
a*b-c
Three address code:
temp=a*b
temp1=temp-c

```
1.assignment
2.arithmetic
3.relational
4.Exit
Enter the choice:2
Enter the expression with arithmetic operator:a/b*c
Three address code:
temp=a/b
temp1=temp*c
1.assignment
2.arithmetic
3.relational
4.Exit
Enter the choice:3
Enter the expression with relational operator
a
<=
b

100 if a<=b goto 103
101 T:=0
102 goto 104
103 T:=1

1.assignment
2.arithmetic
3.relational
4.Exit
Enter the choice:4
```

Conclusion:

Question:

1) **Write Three Address Code for below code.**

```
do
  {  i=i+1;
   } while( a[i] < n);
```

Practical 8

Aim: Write a Parser using yacc or utility of UNIX for following:
Write a Program for a simple desk calculator using YACC Specification.

Software Required:

- Red Hat LINUX
- Turbo C for Linux.

Knowledge Required:

- knowledge of writing regular expressions

Theory/Logic:

- The grammar for creating a four-function calculator is small and simple in its abstract representation.
    1. list: expr \n
       list expr \n
    - expr: NUMBER
      expr + expr
      expr - expr
      expr * expr
      expr / expr
      (expr)
- YACC is a parser generator that is program for converting a grammatical specification of a language like the one above into a parser that will parse statements in the language. YACC provides a way to associate meanings with the components of the grammar in such a way that as the parsing takes place, the meaning can be 'evaluated' as well. The stages in using YACC are the following.
- First, a grammar is written, like the one above, but more precise. This specifies the syntax of the language. YACC can be used at this stage to warn of errors and ambiguities in the grammar.
- Second, each rule of production of the grammar can be augmented with an action a statement of what to do when an instance of that grammatical form is found in a program being parsed. The 'what to do' part is written in C, with conventions for

connecting the grammar to the C code. This defines the semantics of the language.

- Third, a lexical analyzer is needed, which will read the input being parsed and break it up into meaningful chunks for the parser. A NUMBER is an example of a lexical chunk that si several characters long; single-character operators like + and * are also chunks. A lexical chunk is traditionally called a token.
- Finally, a controlling routine is needed, to call the parser that YACC built.

Source Code:-

FILE : hoc.y

```
%{
#define YYSTYPE double      /* data type of yacc stack */
%}
%token NUMBER
%left '+'  '-'    /* left associative, same precedence */
%left '*'  '/'    /* left associative, higher precedence */
%%
list:             /* nothing */
        | list '\n'
        | list expr '\n'   { printf("\t%.8g\n",$2); }
        expr:     NUMBER     { $$ = $1; }
        | expr '+' expr { $$ = $1 + $3; }
        | expr '-' expr { $$ = $1 - $3; }
        | expr '*' expr { $$ = $1 * $3; }
        | expr '/' expr { $$ = $1 / $3; }
        | '(' expr ')'     { $$ = $2; }
        ;
%%
                /* end of grammar */
#include <stdio.h>
#include <ctype.h>
```

```
char *progname;        /* for error message */
int lineno = 1;

main(argc, argv)       /* hoc1 */
char *agrv[];
{
        progname = argv[0];
        yyparse();
}
yylex()          /* Continuing in hoc.y */
{       int c;
        while ( ( c=getchar() )  = = ‘ ‘ || c = = ‘\t’ );
        if ( c = = EOF )
                return 0;
        if ( c = = ‘.’ || isdigit ( c ) )      /* number */
        {       ungetc(c, stdin);
                scanf(“%lf”, &yylval);
                return NUMBER;
        }
        if ( c = = ‘\n’ )
                lineno++;
        return c;
}
yyerror(s)       /* called for yacc syntax error */
        char *s;
{       warning( s, ( char * )  0);       }
warning(s, t)   /* print warning message */
        cahr *s, *t;
{       fprintf(stderr, “%s: %s” , progname, s);
        if ( t )
                fprintf(stderr, “  %s”, t);
```

```
    fprintf(stderr, "  near line %d \n", lineno);
}
```

Output:

Compilation of a yacc program is a two-strp process:

```
$ yacc hoc.y
$ cc y.tab.c –o hoc1
$ hoc1
2/3
0.66666667
-3-4
hoc1: syntax error near line 1
```

Application:

- Used for generating C code by just inputing the grammar to YACC.

Advantages:

- For easily constructing a parser from input a grammar.
- Syntax directed translator can be formed extending a predictive grammar.

Questions:

1. **What is fullform of YACC?**

→ The fullform of YACC is Yet Another Compiler Compiler.

2. **What is YACC compiler?**

→ YACC is a parser generator that is program for converting a grammatical specification of a language like the one above into a parser that will parse statements in the language. YACC provides a way to associate meanings with the components of the grammar in such a way that as the parsing takes place, the meaning can be 'evaluated' as well.

Practical 9

Aim: Demonstrate the parsing technique using ANTLR Tool.

Software Required:

- ANTLR Tool

Knowledge Required:

- Parser concepts

Theory/Logic:

ANTLR, ANother Tool for Language Recognition, (formerly PCCTS) is a language tool that provides a framework for constructing recognizers, compilers, and translators from grammatical descriptions containing Java, C#, Python, or C++ actions. ANTLR is popular because it is easy to understand, powerful, flexible, generates human-readable output, and comes with complete source. ANTLR provides excellent support for tree construction, tree walking, and translation. There are currently over 5000 ANTLR source downloads a month.

Terence Parr is the maniac behind ANTLR and has been working on ANTLR since 1989. He is a professor of computer science at the University of San Francisco. Together with his colleagues, Terence has made a number of fundamental contributions to parsing theory and language tool construction, leading to the resurgence of LL(k)-based recognition tools.

Steps:

Create a new file named Calculator.g4 with the following code

```
grammar Calculator;

operation
    : left=NUMBER operator='+' right=NUMBER
    | left=NUMBER operator='-' right=NUMBER
    | left=NUMBER operator='*' right=NUMBER
    | left=NUMBER operator='/' right=NUMBER
    ;

NUMBER
   : ('0' .. '9') + ('.' ('0' .. '9') +)?
   ;

WS : (' ' | '\t')+ -> channel(HIDDEN);
```

Question:

1. **Define the grammar for arithmetic calculator in AntLR.**